

From Low-Level Memory Internals to High-Performance Modern C++ (C++11 through C++20/23)

FOCUS: RAII, Memory, STL, OOP, Templates

04 Polymorphism, Virtual Tables & Object Layout

Static vs Dynamic Polymorphism

Theory

Category	Mechanism	Binding Time	Cost
Static	Templates, Function Overloading	Compile-time	Zero runtime overhead, larger binary (code bloat)
Dynamic	Virtual Functions, Inheritance	Runtime (vtable dispatch)	1 pointer indirection per call, 1 vptr per instance

```
class Shape {
public:
    virtual void draw() const = 0; // Pure virtual → Abstract class
    virtual ~Shape() = default;    // ALWAYS virtual destructor!
};
class Circle : public Shape {
public:
    void draw() const override { std::cout << "Circle"; }
};
```

Mandatory Virtual Destructor: If deleting a derived object via a base pointer (`Shape* s = new Circle(); delete s;`), the base destructor MUST be `virtual`. Otherwise, only the base destructor runs, causing UB and memory leaks!

Virtual Table (vtable) & vptr Internals

Deep Dive

How dynamic dispatch works under the hood at machine level:

Instance in Memory: Circle Object

bytes pointer | m_radius (8 bytes double)

▼ Virtual Method Table (vtable) [Read-Only]

Data Segment | [0]: &Circle::~Circle() (Destructor address) | [1]: &Circle::draw() (Overridden draw) | [2]: &Shape::area() (Inherited base func)

Invocation: `ptr->draw() ==> (* (ptr->_vptr[1]))(ptr);`

Diamond Problem Resolution: When Class D inherits from B and C, both inheriting from A, use `class B : virtual public A` to guarantee only one shared instance of Base A exists in memory!

05 RAII & Smart Pointer Architecture

std::unique_ptr

Sole Ownership

Zero-cost abstraction. Unique, non-copyable ownership. Automatically deletes object when falling out of scope.

```
// Construction (C++14):
auto p = std::make_unique<Widget>(10);

// Cannot copy!
// auto p2 = p; // Compile Error!

// Ownership transfer:
auto p2 = std::move(p); // p is now null
p2->execute();
```

Cost: Exactly identical to raw pointer in size and runtime efficiency!

std::shared_ptr

Shared Reference Counting

Multiple owners manage single heap object via a heap-allocated **Control Block**.

```
auto s1 = std::make_shared<Node>(42);
auto s2 = s1; // Ref count = 2
// Object freed when last shared_ptr dies
```

shared_ptr s1 — shared_ptr s2 —> [Control Block]
- Strong Count: 2 - Weak Count: 0 - Managed Object ptr

Always use `make_shared` to allocate object & control block in one contiguous chunk.

std::weak_ptr

Breaking Cycles

Non-owning observer. Does not increment strong reference count. Used to break circular dependencies.

```
class Node {
public:
    std::shared_ptr<Node> next;
    std::weak_ptr<Node> prev; // Break cycle!
};

if (auto locked = w.lock()) {
    locked->doWork(); // Safely promoted
} else {
    // Object already destroyed!
}
```

06 Generic Programming: Templates & C++20 Concepts

Function & Class Templates

Generics

```
// Generic Pair Class
template <typename T, typename U = int>
class Pair {
public:
    T first;
    U second;
    Pair(T f, U s) : first(f), second(s) {}
};

// Variadic Template with Fold Expression (C++17)
template <typename... Args>
auto sumAll(Args... args) {
    return (... + args); // Unary right fold: (1 + (2 + 3))
}
```

C++20 Concepts & Constraints

Modern

Replaces cryptic template compiler errors with readable type contract constraints:

```
#include <concepts>

// Define custom concept
template <typename T>
concept Numeric = std::is_arithmetic_v<T>;

// Constrained Template Function
template <Numeric T>
T multiply(T a, T b) {
    return a * b;
}

// multiply("hello", "world"); // Clean compile error!
```

07 The Standard Template Library (STL) Architecture

STL Container Taxonomy & Time Complexity

Complexity

Container	Internal Structure	Access	Insert / Erase
<code>std::vector</code>	Dynamic contiguous array	$O(1)$ random	$O(1)$ back amortized, $O(n)$ middle
<code>std::deque</code>	Array of fixed-size chunks	$O(1)$ random	$O(1)$ at front & back
<code>std::list</code>	Doubly linked list	$O(n)$ linear	$O(1)$ given iterator
<code>std::map</code>	Red-Black Tree (Self-balancing BST)	$O(\log n)$	$O(\log n)$ (keys sorted)
<code>std::unordered_map</code>	Hash Table (buckets & chaining)	$O(1)$ avg, $O(n)$ worst	$O(1)$ avg, $O(n)$ worst

Default Recommendation: Default to `std::vector` for 90% of use cases. Contiguous memory provides unbeatable CPU cache locality compared to pointer-chasing node containers.

Lambda Expressions & Essential STL Algorithms

Functional

Lambda Syntax: `[capture](parameters) mutable → return_type { body }`

- `[]` No captures | `[=]` Capture all by value | `&` Capture all by reference
- `[x, &y]` Explicit capture: `x` by value, `y` by ref

```
std::vector<int> v = {5, 2, 8, 1, 9};

// 1. std::sort with lambda comparator
std::sort(v.begin(), v.end(), [](int a, int b) { return a > b; });

// 2. std::find_if
auto it = std::find_if(v.begin(), v.end(), [](int n) { return n % 2 == 0; });

// 3. std::transform (map operation)
std::vector<int> sq(v.size());
std::transform(v.begin(), v.end(), sq.begin(), [](int x) { return x * x; });

// 4. std::accumulate (reduce / fold)
int sum = std::accumulate(v.begin(), v.end(), 0);
```

08 Modern Concurrency & Thread Synchronization

Thread Management & Data Race Prevention

Threading

```
#include <thread>
#include <mutex>

std::mutex g_mtx;
int counter = 0;

void safeIncrement() {
    // RAII Lock: locks on init, unlocks automatically on exit
    std::lock_guard<std::mutex> lock(g_mtx);
    ++counter;
}

int main() {
    std::thread t1(safeIncrement);
    std::thread t2(safeIncrement);
    t1.join(); // Block until t1 finishes
    t2.join();
}
```

Atomics, Async & Thread Safety Checklist

Lock-Free

- `std::atomic<T>`: Lock-free primitive operations backed by hardware atomic CPU instructions (e.g. `lock_xadd`, `CMPXCHG`).
- `std::async`: Launches task asynchronously, returns a `std::future<T>` that blocks on `.get()`.
- Deadlock Prevention:** Always acquire multiple mutexes in identical order across all threads, or use `std::scoped_lock lock(mtx1, mtx2);` (C++17 deadlock-free lock).

```
std::atomic<int> atomicCounter{0};
atomicCounter.fetch_add(1, std::memory_order_relaxed);

// Async task with future
auto fut = std::async(std::launch::async, []() { return 42; });
int result = fut.get(); // Retrieves value
```

09 Critical Pitfalls, Undefined Behavior & Quick Reference

Memory Pitfalls

Critical

- Dangling Pointer:** Accessing memory after `delete ptr;`. Fix: Set `ptr = nullptr;` or use smart pointers.
- Double Free:** Calling delete twice on the same address. Causes heap corruption crash.
- Memory Leak:** Allocating via `new` without matching `delete` before losing scope.
- Iterator Invalidation:** Modifying a vector (e.g. `push_back`) while iterating through it may trigger reallocation, invalidating all existing iterators!

Undefined Behavior (UB)

Danger

- Dereferencing a null or uninitialized pointer (`*nullptr`).
- Signed integer overflow (e.g. `INT_MAX + 1` is UB in C++).
- Out-of-bounds array indexing (`arr[10]` on size 5). Use `v.at(i)` for bounds checking.
- Returning a pointer/reference to a local stack variable.
- Data races (two concurrent threads accessing memory where at least one writes without synchronization).

Modern C++ Best Practice Rules

Summary

- Never use raw `new` / `delete`; wrap in `std::unique_ptr`.
- Pass large read-only objects by `const T&`.
- Mark single-argument constructors `explicit` to prevent implicit conversions.
- Use `nullptr`, never `NULL` or `0`.
- Enable compiler warnings: `-Wall -Wextra -Wpedantic -Werror`.
- Run Sanitizers: `-fsanitize=address,undefined`.

Modern C++ Standards Quick Reference Matrix

Evolution

Standard	Year	Major Highlights
C++11	2011	Move semantics, auto, lambdas, smart pointers, nullptr, constexpr, thread support
C++14	2014	Generic lambdas, return type deduction, make_unique, relaxed constexpr
C++17	2017	Structured bindings, std::optional, std::variant, std::string_view, if constexpr, fold expressions
C++20	2020	Concepts, Ranges, Coroutines, Modules, spaceship operator <=>, consteval, std::format
C++23	2023	std::print, deducing this, std::expected, multidimensional subscript operator []